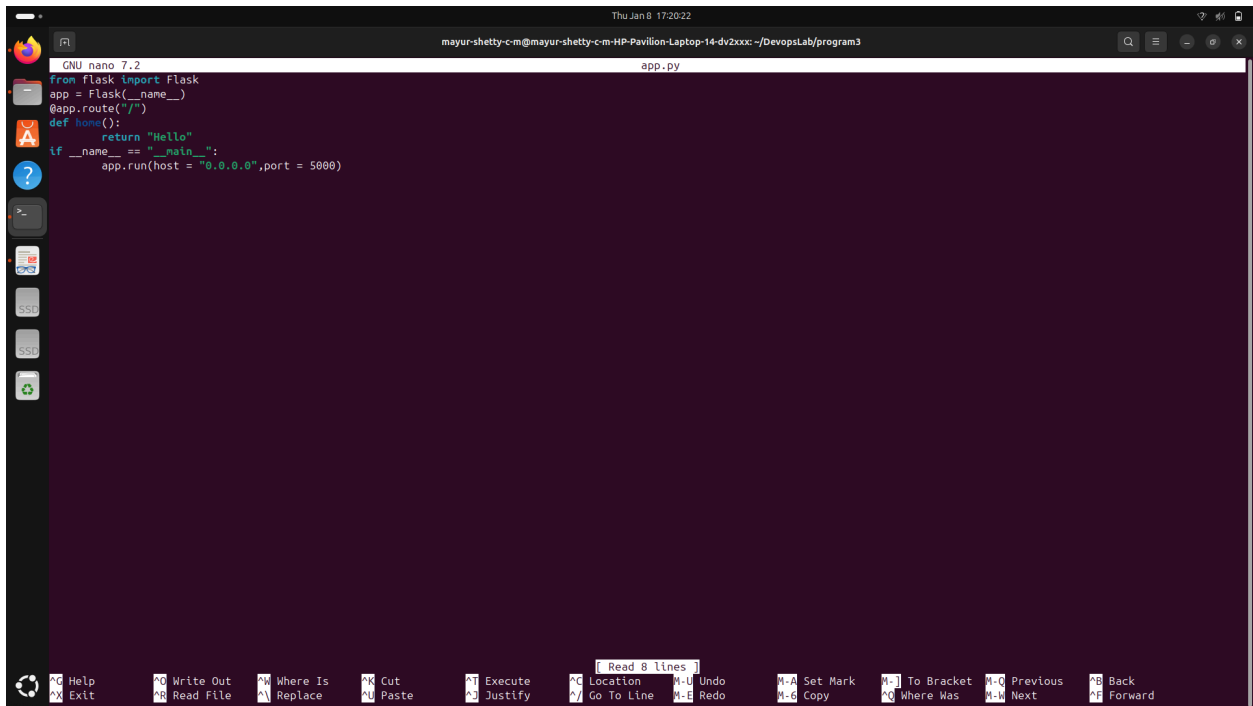


3. Code a Dockerized Python Flask or Node.js Application

Program3

-Dockerfile

-[app.py](#)



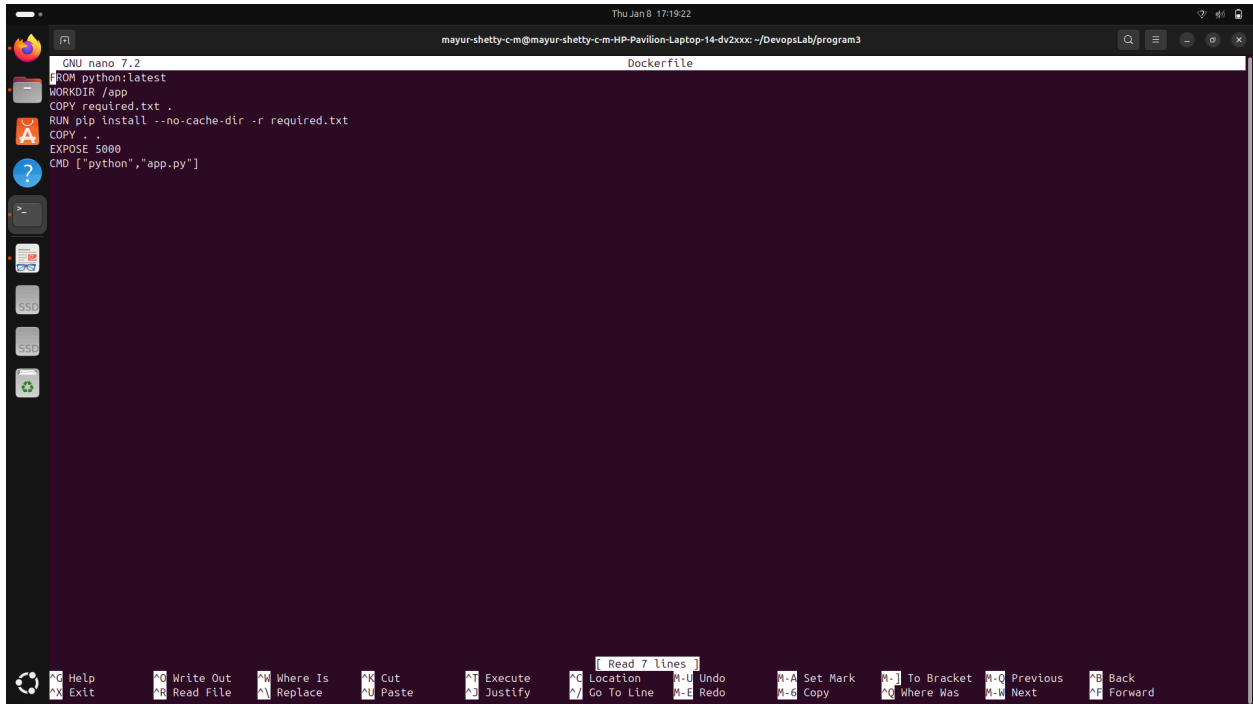
```
GNU nano 7.2 app.py
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Hello"
if __name__ == "__main__":
    app.run(host = "0.0.0.0", port = 5000)
```

-required.txt

Dockerfile

Step 1: Create a Docker file and save this code

Nano Dockerfile



The screenshot shows a terminal window titled "GNU nano 7.2" with a dark purple background. The terminal content is a Dockerfile with the following lines: FROM python:latest, WORKDIR /app, COPY required.txt ., RUN pip install --no-cache-dir -r required.txt, COPY . ., EXPOSE 5000, and CMD ["python","app.py"]. The terminal window has a sidebar on the left with icons for file manager, terminal, and other applications. The bottom status bar shows various keyboard shortcuts for nano editor operations.

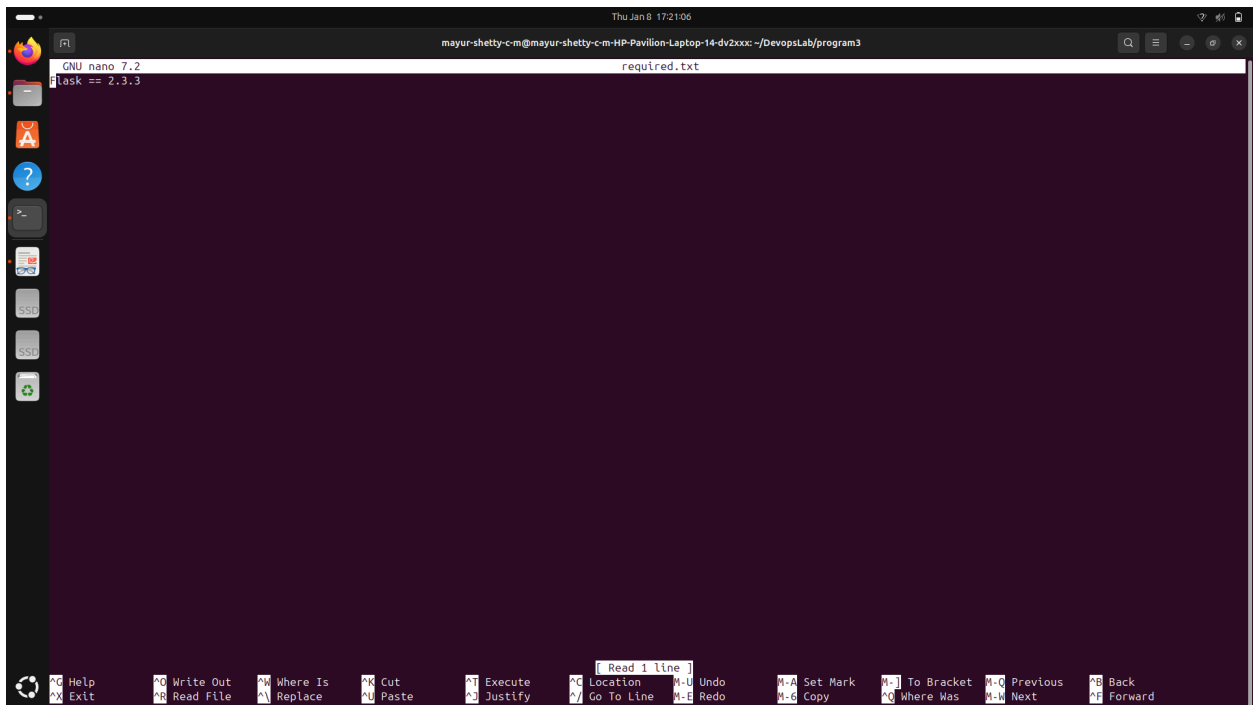
```
GNU nano 7.2
FROM python:latest
WORKDIR /app
COPY required.txt .
RUN pip install --no-cache-dir -r required.txt
COPY . .
EXPOSE 5000
CMD ["python","app.py"]
```

Step 2: Create a [app.py](#) file and save this code

Nano [app.py](#)

Step 3: Create a required.txt file and save this code

Nano required.txt



The screenshot shows a terminal window titled "GNU nano 7.2" with a dark purple background. The terminal content is a file named "required.txt" containing the line "flask == 2.3.3". The terminal window has a sidebar on the left with icons for file manager, terminal, and other applications. The bottom status bar shows various keyboard shortcuts for nano editor operations.

```
GNU nano 7.2
required.txt
flask == 2.3.3
```

Step4: Build a docker file

Docker build -t program3 .

Step 5: Run the container in a specific port number

Docker run -p 5000:5000 program3

